Axion Mechanics

Axion Mechanics List (Fresh)

1) File System Mechanics

- **Repo file tree contract** (where everything lives)
- **Storage rules** (what is immutable vs overwritable)
- **Atomic write rules** (no partial/corrupt files)
- **Naming + versioning rules** (consistent IDs + filenames)

2) Configuration Mechanics

- **config/axion.config.json** (pins paths, versions, policies)
- **Version pin model** (what versions are active)
- **Policy model** (stop-on-fail, unknowns, overrides allowed/denied)

3) Identity & Referencing Mechanics

- **ID format rules** (artifact IDs + spec IDs)
- **Pointer model** (how errors point to exact locations)
- **Reference resolution rules** (what can reference what; must-resolve policy)

4) Intake Mechanics

- **Intake schema contract**
- **Intake ruleset contract**
- **Enum registry contract**

- **Validation report format**
- **Normalizer rules** (how raw input is normalized)

5) Canonical Truth Mechanics

- **Canonical spec schema**
- **Canonical spec builder rules (intake → spec)**
- **Canonical validation rules** (IDs unique, refs resolve)
- **Unknown handling contract** (if enforced)

6) Standards Mechanics

- **Standards index contract**
- **Applicability map contract**
- **Constraint packs contract**
- **Resolved standards snapshot format**
- **Resolver order rules** (deterministic)

7) Template System Mechanics (format + enforcement only)

- **Template index/registry contract**
- **Template selection result format**
- **Render output/envelope format**
- **Template completeness rules**
- **Placeholder/derive syntax rules** (the allowed forms)

8) Planning Mechanics

- **Work breakdown format**
- **Acceptance map format**
- **Build plan format**
- **Coverage rules** (what must be covered for a run to be “complete”)

9) Pipeline Orchestration Mechanics

- **Pipeline definition** (stages + order)
- **Stage IO contracts** (consumes/produces)
- **Gate model** (gate_id, pass condition, evidence refs)
- **Stage report format**
- **Run manifest format**

10) Verification & Proof Mechanics

- **Gate report format**
- **Proof ledger format**
- **Proof types rules**
- **Verification command policy**
- **Completion criteria** (unit done + run done)

11) State & Continuity Mechanics

- **State snapshot format**

- **Resume rules** (what's required to resume)
- **Run diff format** (what changed between runs)

12) Locking & Governance Mechanics

- **Lock record format** (hash + immutability marker)
- **Version registry format**
- **Changelog format**
- **Override record format** (only if overrides allowed)
- **Compatibility notes format** (how versions interoperate)

13) Packaging & Handoff Mechanics

- **Folder tree spec** (kit structure)
- **Kit manifest/index formats**
- **Packaging manifest format** (hash inventory)
- **Entrypoint contract** (what to read first; reading order)
-

File System Mechanics

File System Mechanics

- **Repo file tree contract** (where everything lives)
- **Storage rules** (what is immutable vs overwritable)
- **Atomic write rules** (no partial/corrupt files)
- **Naming + versioning rules** (consistent IDs + filenames)

REPO_FILE_TREE_CONTRACT.v1.jso
n

```
{
  "artifact_type": "REPO_FILE_TREE_CONTRACT",
  "contract_version": "1.0.0",
  "root": "axion/",
  "required_paths": [
    "axion/system_docs/",
    "axion/knowledge/",
    "axion/config/",
    "axion/artifacts/",
    "axion/scripts/",
    "axion/runs/"
  ],
  "system_docs_structure": {
    "root": "axion/system_docs/",
    "must_contain_folders": ["SYS", "INT", "CAN", "STD", "TMP", "PLAN", "VER", "KIT", "STATE",
"GOV", "EXEC"],
    "rule": "All official system docs live under system_docs/ only."
  },
  "knowledge_structure": {
    "root": "axion/knowledge/",
    "rule": "Knowledge is input-only. Runs may read and cite usage, but never edit."
  },
  "config_structure": {
    "root": "axion/config/",
    "required_files": ["axion.config.json"]
  },
  "artifacts_structure": {
    "root": "axion/artifacts/",
    "must_contain_folders": [
      "intake",
      "canonical",
      "standards",
      "templates",
      "pipeline",
      "planning",
      "packaging",
      "state_gov",
      "verification"
    ],
    "rule": "Compiled contracts only. No per-run outputs stored here."
  },
  "runs_structure": {
    "root": "axion/runs/",
    "run_dir_pattern": "^RUN-[0-9]{4,}$",
```

```
    "required_run_subpaths": [
      "run_manifest.json",
      "intake/",
      "canonical/",
      "standards/",
      "docs_rendered/",
      "planning/",
      "stage_reports/",
      "gate_reports/",
      "proofs/",
      "package/"
    ],
    "rule": "All generated outputs live under runs/ only."
  }
}
```

STORAGE_RULES.v1.json

{
  "artifact_type": "STORAGE_RULES",
  "rules_version": "1.0.0",
  "immutability_zones": [
    {
      "zone_id": "Z1_SYSTEM_DOCS",
      "path_prefix": "axion/system_docs/",
      "mutable": true,
      "rule": "Docs may be edited by humans, but never edited by runs."
    },
    {
      "zone_id": "Z2_KNOWLEDGE",
      "path_prefix": "axion/knowledge/",
      "mutable": true,
      "rule": "Humans may update knowledge; runs must treat it read-only."
    },
    {
      "zone_id": "Z3_CONFIG",
      "path_prefix": "axion/config/",
      "mutable": true,
      "rule": "Humans may update config; runs must record the config hash they used."
    },
    {
      "zone_id": "Z4_ARTIFACTS_COMPILED",
      "path_prefix": "axion/artifacts/",
      "mutable": true,
      "rule": "Compiled contracts may be updated only via version bump (see NAMING_VERSIONING_RULES)."
    },
    {
      "zone_id": "Z5_RUN_OUTPUTS",
      "path_prefix": "axion/runs/",
      "mutable": true,
      "rule": "Run outputs are append-only per run_id. A run_id folder is never overwritten; new run uses new run_id."
    }
  ],
  "overwrite_policy": {
    "allowed": [
      { "path_prefix": "axion/runs/RUN-*/", "scope": "within_run_temp_only", "rule": "Temp files may be overwritten; final artifacts/reports must be written once." }
    ],
    "forbidden": [

{ "path_prefix": "axion/runs/RUN-*/run_manifest.json", "rule": "Run manifest is write-once; updates require a new file name with a version suffix or an append-only log field." }
]
}
}

ATOMIC_WRITE_RULES.v1.json

```
{
  "artifact_type": "ATOMIC_WRITE_RULES",
  "rules_version": "1.0.0",
  "protocol": {
    "step_order": [
      "write_temp_file",
      "fsync_temp_file",
      "rename_temp_to_final",
      "fsync_parent_dir"
    ],
    "temp_suffix": ".tmp",
    "rule": "All JSON/MD outputs written by scripts must use atomic rename."
  },
  "crash_safety": {
    "rule": "On startup, the system must scan for orphan .tmp files and either resume or delete
them based on run_id ownership."
  },
  "prohibited_patterns": [
    "in-place overwrite of final files",
    "partial streaming writes to final paths without temp"
  ]
}
```

NAMING_VERSIONING_RULES.v1.jso
n

```
{
  "artifact_type": "NAMING_VERSIONING_RULES",
  "rules_version": "1.0.0",

  "artifact_filename_pattern": "^[A-Z0-9_]+\\.v[0-9]+\\.[0-9]+\\.[0-9]+\\.(json|md)$",
  "artifact_examples": [
    "INTAKE_SCHEMA.v1.0.0.json",
    "PIPELINE_DEFINITION.v1.0.0.json",
    "GATE_REPORT_FORMAT.v1.0.0.json"
  ],

  "run_dir_pattern": "^RUN-[0-9]{4,}$",

  "spec_ref_pattern": "^[a-z_]+:[A-Z]{2,6}-[0-9]{4}$",
  "spec_ref_examples": [
    "screen:SCR-0001",
    "endpoint:API-0001",
    "entity:ENT-0001",
    "business_rule:BR-0001",
    "component:CMP-0001",
    "acceptance_criterion:AC-0001",
    "test_case:TC-0001",
    "flow:FLW-0001"
  ],

  "version_bump_rules": [
    {
      "rule_id": "VER-001",
      "when": "any change to meaning, required fields, enums, or constraints",
      "then": "bump artifact version"
    },
    {
      "rule_id": "VER-002",
      "when": "format-only change that does not alter meaning",
      "then": "bump patch version"
    }
  ],

  "path_conventions": {
    "system_docs": "axion/system_docs/{FAMILY}/{DOC_ID}.md",
    "compiled_artifacts": "axion/artifacts/{AREA}/{ARTIFACT_NAME}.vX.Y.Z.json",
    "run_outputs": "axion/runs/{RUN_ID}/..."
  }
}
```

Configuration Mechanics

Configuration Mechanics

- **config/axion.config.json** (pins paths, versions, policies)
- **Version pin model** (what versions are active)
- **Policy model** (stop-on-fail, unknowns, overrides allowed/denied)

config/axion.config.json

```
{
  "config_version": "1.0.0",

  "paths": {
    "root": "axion/",
    "system_docs": "axion/system_docs/",
    "knowledge": "axion/knowledge/",
    "artifacts": "axion/artifacts/",
    "runs": "axion/runs/",
    "scripts": "axion/scripts/"
  },

  "active_versions": {
    "intake": {
      "INTAKE_SCHEMA": "1.0.0",
      "INTAKE_RULESET": "1.0.0",
      "ENUM_REGISTRY": "1.0.0",
      "VALIDATION_REPORT_FORMAT": "1.0.0"
    },
    "canonical": {
      "CANONICAL_SPEC_SCHEMA": "1.0.0",
      "REFERENCE_RULES_POINTER_MODEL": "1.0.0",
      "CANONICAL_ID_REFERENCE_RULES": null,
      "UNKNOWN_POLICY": null,
      "UNKNOWN_REGISTRY_FORMAT": null
    },
    "standards": {
      "STANDARDS_INDEX": "1.0.0",
      "APPLICABILITY_MAP": "1.0.0",
      "CONSTRAINT_PACKS": "1.0.0",
      "RESOLVED_STANDARDS_SNAPSHOT_FORMAT": null
    },
    "templates": {
      "TEMPLATE_INDEX_REGISTRY": "1.0.0",
      "TEMPLATE_COMPLETENESS_RULES": "1.0.0",
      "TEMPLATE_RENDER_OUTPUT_FORMAT": "1.0.0",
      "TEMPLATE_SELECTION_RESULT_FORMAT": null
    },
    "pipeline": {
      "PIPELINE_DEFINITION": "1.0.0",
      "STAGE_IO_CONTRACTS": "1.0.0",
      "STAGE_REPORT_FORMAT": "1.0.0",
      "RUN_MANIFEST_FORMAT": "1.0.0"
    },
```

```
  "planning": {
    "WORK_BREAKDOWN_FORMAT": "1.0.0",
    "ACCEPTANCE_MAP_FORMAT": "1.0.0",
    "BUILD_PLAN_FORMAT": "1.0.0",
    "COVERAGE_RULES": "1.0.0"
  },
  "packaging": {
    "FOLDER_TREE_SPEC": "1.0.0",
    "KIT_MANIFEST_FORMAT": "1.0.0",
    "PACKAGING_MANIFEST_FORMAT": "1.0.0"
  },
  "state_gov": {
    "STATE_SNAPSHOT_FORMAT": "1.0.0",
    "VERSION_REGISTRY_FORMAT": "1.0.0",
    "CHANGELOG_FORMAT": "1.0.0",
    "LOCK_RECORD_FORMAT": "1.0.0",
    "OVERRIDE_RECORD_FORMAT": null
  },
  "verification": {
    "GATE_REPORT_FORMAT": "1.0.0",
    "PROOF_LEDGER_FORMAT": "1.0.0",
    "VERIFICATION_COMMAND_POLICY": "1.0.0",
    "PROOF_TYPES_RULES": "1.0.0",
    "COMPLETION_CRITERIA": "1.0.0"
  }
},

"policies": {
  "stop_on_fail": true,
  "warnings_allowed": true,

  "unknowns": {
    "enabled": false,
    "allow_unknown_tokens_in_docs": false,
    "require_unknown_registry": false
  },

  "overrides": {
    "allowed": false,
    "require_override_record": false
  },

  "determinism": {
    "resolver_order_locked": true,
```

"atomic_writes_required": true,
"no_network_calls": true
}
}
}

VERSION_PIN_MODEL.v1.0.0.json

{
  "artifact_type": "VERSION_PIN_MODEL",
  "model_version": "1.0.0",
  "rules": [
    "A run MUST record the exact config_version and a hash of config/axion.config.json.",
    "active_versions values are semantic versions for compiled artifacts.",
    "null means 'not used in this system version' and MUST NOT be required by gates."
  ],
  "pin_fields": {
    "active_versions": {
      "intake": ["INTAKE_SCHEMA", "INTAKE_RULESET", "ENUM_REGISTRY", "VALIDATION_REPORT_FORMAT"],
      "canonical": ["CANONICAL_SPEC_SCHEMA", "REFERENCE_RULES_POINTER_MODEL", "CANONICAL_ID_REFERENCE_RULES", "UNKNOWN_POLICY", "UNKNOWN_REGISTRY_FORMAT"],
      "standards": ["STANDARDS_INDEX", "APPLICABILITY_MAP", "CONSTRAINT_PACKS", "RESOLVED_STANDARDS_SNAPSHOT_FORMAT"],
      "templates": ["TEMPLATE_INDEX_REGISTRY", "TEMPLATE_COMPLETENESS_RULES", "TEMPLATE_RENDER_OUTPUT_FORMAT", "TEMPLATE_SELECTION_RESULT_FORMAT"],
      "pipeline": ["PIPELINE_DEFINITION", "STAGE_IO_CONTRACTS", "STAGE_REPORT_FORMAT", "RUN_MANIFEST_FORMAT"],
      "planning": ["WORK_BREAKDOWN_FORMAT", "ACCEPTANCE_MAP_FORMAT", "BUILD_PLAN_FORMAT", "COVERAGE_RULES"],
      "packaging": ["FOLDER_TREE_SPEC", "KIT_MANIFEST_FORMAT", "PACKAGING_MANIFEST_FORMAT"],
      "state_gov": ["STATE_SNAPSHOT_FORMAT", "VERSION_REGISTRY_FORMAT", "CHANGELOG_FORMAT", "LOCK_RECORD_FORMAT", "OVERRIDE_RECORD_FORMAT"],
      "verification": ["GATE_REPORT_FORMAT", "PROOF_LEDGER_FORMAT", "VERIFICATION_COMMAND_POLICY", "PROOF_TYPES_RULES", "COMPLETION_CRITERIA"]
    }
  }
}

POLICY_MODEL.v1.0.0.json

```
{
  "artifact_type": "POLICY_MODEL",
  "model_version": "1.0.0",

  "policy_fields": {
    "stop_on_fail": { "type": "boolean", "default": true },
    "warnings_allowed": { "type": "boolean", "default": true },

    "unknowns": {
      "enabled": { "type": "boolean", "default": false },
      "allow_unknown_tokens_in_docs": { "type": "boolean", "default": false },
      "require_unknown_registry": { "type": "boolean", "default": false }
    },

    "overrides": {
      "allowed": { "type": "boolean", "default": false },
      "require_override_record": { "type": "boolean", "default": false }
    },

    "determinism": {
      "resolver_order_locked": { "type": "boolean", "default": true },
      "atomic_writes_required": { "type": "boolean", "default": true },
      "no_network_calls": { "type": "boolean", "default": true }
    }
  },

  "invariants": [
    "If overrides.allowed == true then require_override_record must be true.",
    "If unknowns.enabled == true then allow_unknown_tokens_in_docs and
require_unknown_registry must be explicitly set (true/false).",
    "If atomic_writes_required == true then ATOMIC_WRITE_RULES must be enforced by
scripts."
  ]
}
```

Identity & Referencing Mechanics

Identity & Referencing Mechanics

- **ID format rules** (artifact IDs + spec IDs)
- **Pointer model** (how errors point to exact locations)
- **Reference resolution rules** (what can reference what; must-resolve policy)

ID_FORMAT_RULES.v1.0.0.json

```
{
  "artifact_type": "ID_FORMAT_RULES",
  "rules_version": "1.0.0",

  "artifact_ids": {
    "artifact_name_pattern": "^[A-Z0-9_]+$",
    "artifact_filename_pattern": "^[A-Z0-9_]+\\.v[0-9]+\\.[0-9]+\\.[0-9]+\\.(json|md)$",
    "examples": [
      "INTAKE_SCHEMA.v1.0.0.json",
      "PIPELINE_DEFINITION.v1.0.0.json",
      "PACKAGING_MANIFEST_FORMAT.v1.0.0.json"
    ]
  },

  "doc_ids": {
    "doc_id_pattern": "^[A-Z]{2,6}-[0-9]{2,3}$",
    "examples": ["SYS-01", "INT-03", "CAN-01", "KIT-02", "EXEC-01"]
  },

  "run_ids": {
    "run_id_pattern": "^RUN-[0-9]{4,}$",
    "examples": ["RUN-0001", "RUN-0125"]
  },

  "spec_ids": {
    "ref_pattern": "^[a-z_]+:[A-Z]{2,6}-[0-9]{4}$",
    "allowed_prefixes": [
      "screen",
      "endpoint",
      "entity",
      "business_rule",
      "component",
      "acceptance_criterion",
      "test_case",
      "flow"
    ],
    "prefix_map": {
      "screen": "SCR",
      "endpoint": "API",
      "entity": "ENT",
      "business_rule": "BR",
      "component": "CMP",
      "acceptance_criterion": "AC",
      "test_case": "TC",
```

```
      "flow": "FLW"
    },
    "examples": [
      "screen:SCR-0001",
      "endpoint:API-0001",
      "entity:ENT-0001",
      "business_rule:BR-0001",
      "component:CMP-0001",
      "acceptance_criterion:AC-0001",
      "test_case:TC-0001",
      "flow:FLW-0001"
    ]
  }
}
```

POINTER_MODEL.v1.0.0.json

```
{
  "artifact_type": "POINTER_MODEL",
  "model_version": "1.0.0",

  "pointer_type": "json_pointer",
  "pointer_format": "RFC6901",

  "rules": [
    "Pointers MUST be absolute JSON Pointer strings (start with '/').",
    "Pointers MUST refer to paths within the relevant payload/artifact being validated.",
    "When an error concerns list membership (duplicate, missing, min_items), pointer SHOULD
target the list container path.",
    "When an error concerns a specific field value, pointer SHOULD target the exact field path.",
    "When multiple locations apply, emit multiple pointers."
  ],

  "examples": {
    "intake": [
      "/routing/category",
      "/spec/must_have_features",
      "/final/confirm_priorities"
    ],
    "canonical_spec": [
      "/spec_index/screen/0/id",
      "/spec_index/endpoint/1/path",
      "/spec_index/entity/0/fields/2/name"
    ],
    "templates": [
      "/content_md",
      "/sections/5/title"
    ]
  }
}
```

REFERENCE_RESOLUTION_RULES.v1.0.0.json

```
{
  "artifact_type": "REFERENCE_RESOLUTION_RULES",
  "rules_version": "1.0.0",

  "must_resolve": true,
  "allow_external_refs": false,

  "ref_target_categories": [
    "screen",
    "endpoint",
    "entity",
    "business_rule",
    "component",
    "acceptance_criterion",
    "test_case",
    "flow"
  ],

  "allowed_reference_edges": [
    { "from": "screen", "to": ["component", "entity", "acceptance_criterion", "flow"] },
    { "from": "endpoint", "to": ["entity", "business_rule", "test_case", "acceptance_criterion"] },
    { "from": "entity", "to": ["entity", "business_rule"] },
    { "from": "business_rule", "to": ["entity", "endpoint", "flow"] },
    { "from": "component", "to": ["screen", "endpoint"] },
    { "from": "acceptance_criterion", "to": ["screen", "endpoint", "flow", "test_case"] },
    { "from": "test_case", "to": ["endpoint", "acceptance_criterion", "business_rule", "flow"] },
    { "from": "flow", "to": ["screen", "endpoint", "entity", "business_rule", "acceptance_criterion",
"test_case", "component"] }
  ],

  "resolution_algorithm": {
    "rule": "A ref is valid if it matches ID_FORMAT_RULES.spec_ids.ref_pattern and the
referenced id exists in the Canonical Spec payload under its declared prefix category.",
    "steps": [
      "parse prefix before ':' (category)",
      "validate category is allowed",
      "validate full ref matches regex",
      "lookup ref existence in spec_index[category]",
      "fail if missing"
    ]
  },

  "failure_codes": {
    "INVALID_REF_FORMAT": "Ref does not match required regex/prefix rules",
```

"UNKNOWN_REF_CATEGORY": "Ref prefix category not allowed",
"UNRESOLVED_REF": "Ref target not found in payload",
"DISALLOWED_EDGE": "Reference edge not allowed by allowed_reference_edges"
}
}

Intake Mechanics

Intake Mechanics

- **Intake schema contract**
- **Intake ruleset contract**
- **Enum registry contract**
- **Validation report format**
- **Normalizer rules** (how raw input is normalized)

INTAKE_SCHEMA.v1.0.0.json

{
  "artifact_type": "INTAKE_SCHEMA",
  "schema_version": "1.0.0",
  "required_top_level": [
    "schema_version",
    "form_version",
    "routing",
    "project",
    "delivery",
    "intent",
    "spec",
    "final"
  ],
  "top_level_sections": [
    "routing",
    "project",
    "inputs",
    "existing",
    "delivery",
    "intent",
    "brand",
    "design",
    "spec",
    "data",
    "auth",
    "integrations",
    "nfr",
    "app_ui",
    "service",
    "infra",
    "sdk",
    "final"
  ],
  "notes": "Structural/type contract only. Conditional requiredness is enforced by INTAKE_RULESET."
}

INTAKE_RULESET.v1.0.0.json

```
{
  "artifact_type": "INTAKE_RULESET",
  "ruleset_version": "1.0.0",
  "rule_format_ref": "artifacts/verification/GATE_REPORT_FORMAT.v1.0.0.json",
  "rules": [
    {
      "rule_id": "INT-ROUTING-01",
      "description": "type_preset must be valid for category",
      "when": { "always": true },
      "then": [{ "op": "preset_valid_for_category", "args": ["routing.category", "routing.type_preset"]
}],
      "error_code": "INVALID_ENUM_FOR_CONTEXT",
      "pointer_paths": ["/routing/category", "/routing/type_preset"],
      "severity": "error"
    },
    {
      "rule_id": "INT-EXIST-01",
      "description": "Existing projects must provide repo link and current state summary",
      "when": { "eq": ["routing.build_target", "existing"] },
      "then": [
        { "op": "required", "path": "/existing/existing_repo_link" },
        { "op": "format", "path": "/existing/existing_repo_link", "format": "uri" },
        { "op": "required", "path": "/existing/current_state_summary" }
      ],
      "error_code": "DEPENDENCY_MISSING",
      "pointer_paths": ["/existing/existing_repo_link", "/existing/current_state_summary"],
      "severity": "error"
    },
    {
      "rule_id": "INT-FINAL-01",
      "description": "Final confirmations must be true",
      "when": { "always": true },
      "then": [
        { "op": "equals", "path": "/final/confirm_priorities", "value": true },
        { "op": "equals", "path": "/final/confirm_out_of_scope", "value": true },
        { "op": "equals", "path": "/final/confirm_constraints", "value": true }
      ],
      "error_code": "REQUIRED",
      "pointer_paths": ["/final/confirm_priorities", "/final/confirm_out_of_scope",
"/final/confirm_constraints"],
      "severity": "error"
    }
  ],
  "notes": "This ruleset is intentionally minimal at v1. Add rules by version bump."
```

}

ENUM_REGISTRY.v1.0.0.json

```
{
  "artifact_type": "ENUM_REGISTRY",
  "enum_registry_version": "1.0.0",
  "enums": {
    "SkillLevel": ["beginner", "intermediate", "expert"],
    "ProjectCategory": ["application", "service", "infrastructure", "library_sdk"],
    "BuildTarget": ["new", "existing"],
    "AudienceContext": ["consumer", "internal", "developer"],
    "DesiredScope": ["mvp", "full", "unsure"],
    "PriorityBias": ["speed", "balanced", "quality"]
  },
  "type_preset": {
    "values": [
      "web_app_public",
      "web_app_internal",
      "mobile_app",
      "desktop_app",
      "api_service",
      "event_driven_service",
      "batch_job",
      "infra_platform",
      "infra_pipeline",
      "sdk_client",
      "sdk_framework"
    ],
    "allowed_by_category": {
      "application": ["web_app_public", "web_app_internal", "mobile_app", "desktop_app"],
      "service": ["api_service", "event_driven_service", "batch_job"],
      "infrastructure": ["infra_platform", "infra_pipeline"],
      "library_sdk": ["sdk_client", "sdk_framework"]
    }
  }
}
```

VALIDATION_REPORT_FORMAT.v1.0.0.json

```
{
  "artifact_type": "VALIDATION_REPORT_FORMAT",
  "report_version": "1.0.0",
  "required_fields": [
    "schema_version",
    "form_version",
    "ruleset_version",
    "routing_snapshot",
    "included_modules",
    "result",
    "errors",
    "warnings"
  ],
  "result_shape": {
    "status": "pass|fail",
    "error_count": "number",
    "warning_count": "number"
  },
  "rule_hit_shape": {
    "rule_id": "string",
    "error_code": "string",
    "severity": "error|warning",
    "message": "string",
    "pointer_paths": "array<string>",
    "details": "object",
    "evidence": {
      "received_value": "any",
      "expected": "any"
    }
  }
}
```

NORMALIZER_RULES.v1.0.0.json

```
{
  "artifact_type": "NORMALIZER_RULES",
  "rules_version": "1.0.0",
  "rules": [
    {
      "rule_id": "NORM-STR-01",
      "description": "Trim leading/trailing whitespace in all string fields",
      "scope": "all_strings",
      "action": "trim"
    },
    {
      "rule_id": "NORM-STR-02",
      "description": "Normalize line endings to LF",
      "scope": "all_strings",
      "action": "normalize_newlines",
      "target": "LF"
    },
    {
      "rule_id": "NORM-URL-01",
      "description": "Normalize URLs by ensuring scheme and stripping trailing spaces",
      "scope": "fields_with_format_uri",
      "action": "normalize_url"
    },
    {
      "rule_id": "NORM-ENUM-01",
      "description": "Lowercase enum tokens where enums are lowercase-defined",
      "scope": "enum_fields",
      "action": "normalize_enum_case",
      "mode": "lowercase"
    },
    {
      "rule_id": "NORM-LIST-01",
      "description": "Remove empty strings from string arrays",
      "scope": "string_arrays",
      "action": "drop_empty_items"
    }
  ],
  "notes": "Normalization must not invent values; only transforms existing user input."
}
```

Canonical Truth Mechanics

Canonical Truth Mechanics

- **Canonical spec schema**
- **Canonical spec builder rules (intake → spec)**
- **Canonical validation rules** (IDs unique, refs resolve)
- **Unknown handling contract** (if enforced)

CANONICAL_SPEC_SCHEMA.v1.0.0.json

```
{
  "artifact_type": "CANONICAL_SPEC_SCHEMA",
  "spec_schema_version": "1.0.0",
  "required_fields": [
    "spec_schema_version",
    "project_id",
    "routing_snapshot",
    "reference_rules_version",
    "spec_index"
  ],
  "spec_index_buckets": [
    "screen",
    "endpoint",
    "entity",
    "business_rule",
    "component",
    "acceptance_criterion",
    "test_case",
    "flow"
  ],
  "notes": "This schema defines required top-level shape + required buckets. Field-level item schemas are validated by the canonical validator rules below."
}
```

CANONICAL_SPEC_BUILDER_RULES
.v1.0.0.json

{
  "artifact_type": "CANONICAL_SPEC_BUILDER_RULES",
  "rules_version": "1.0.0",
  "inputs": ["NORMALIZED_INPUT_RECORD", "RESOLVED_STANDARDS_SNAPSHOT"],
  "outputs": ["CANONICAL_SPEC"],

  "id_allocation_policy": {
    "deterministic": true,
    "sequence_per_bucket": true,
    "seed_fields": ["project_id"],
    "format_ref": "artifacts/identity/ID_FORMAT_RULES.v1.0.0.json"
  },

  "mapping_rules": [
    {
      "rule_id": "MAP-ROLES-01",
      "description": "Spec roles in intake become entities or actors in screens/flows, depending on routing.category",
      "from": "/spec/roles",
      "to": "spec_index.flow[].involves or spec_index.screen[].actors",
      "mode": "derive"
    },
    {
      "rule_id": "MAP-FEATURES-01",
      "description": "Must-have features become flows (minimum) and/or screens/endpoints based on category",
      "from": "/spec/must_have_features",
      "to": "spec_index.flow",
      "mode": "derive"
    },
    {
      "rule_id": "MAP-ACCEPTANCE-01",
      "description": "Acceptance criteria are created later by planner; canonical spec carries placeholders only if present",
      "from": "/final/acceptance_criteria",
      "to": "spec_index.acceptance_criterion",
      "mode": "optional"
    }
  ],

  "notes": [
    "Builder rules must not invent requirements; when needed data is missing, emit Unknowns only if UNKNOWN_POLICY enables it.",

"For v1, canonicalization may output minimal buckets; coverage enforcement is done by COVERAGE_RULES in planning/gates."
]
}

CANONICAL_VALIDATION_RULES.v1.0.0.json

```
{
  "artifact_type": "CANONICAL_VALIDATION_RULES",
  "rules_version": "1.0.0",
  "depends_on": [
    "ID_FORMAT_RULES.v1.0.0.json",
    "REFERENCE_RESOLUTION_RULES.v1.0.0.json",
    "POINTER_MODEL.v1.0.0.json"
  ],
  "rules": [
    {
      "rule_id": "CAN-ID-01",
      "description": "All spec item ids must match spec id format",
      "check": "all_ids_match_regex",
      "error_code": "INVALID_REF_FORMAT",
      "severity": "error",
      "pointer_paths": ["/spec_index"]
    },
    {
      "rule_id": "CAN-ID-02",
      "description": "All spec item ids must be globally unique across buckets",
      "check": "global_uniqueness",
      "error_code": "DUPLICATE_VALUE",
      "severity": "error",
      "pointer_paths": ["/spec_index"]
    },
    {
      "rule_id": "CAN-REF-01",
      "description": "All refs must resolve within the canonical spec payload",
      "check": "all_refs_resolve",
      "error_code": "UNRESOLVED_REF",
      "severity": "error",
      "pointer_paths": ["/spec_index"]
    },
    {
      "rule_id": "CAN-REF-02",
      "description": "Reference edges must be allowed by allowed_reference_edges",
      "check": "allowed_edges_only",
      "error_code": "DISALLOWED_EDGE",
      "severity": "error",
      "pointer_paths": ["/spec_index"]
    }
  ],
  "notes": "Validator emits a Gate Report using GATE_REPORT_FORMAT."
}
```

UNKNOWN_POLICY.v1.0.0.json

{
  "artifact_type": "UNKNOWN_POLICY",
  "policy_version": "1.0.0",
  "enabled": false,
  "rules": [
    "If enabled=false, canonical builder and template renderer must not emit UNKNOWN tokens.",
    "If enabled=true, every unknown must be recorded in UNKNOWN_REGISTRY and referenced by unknown_id."
  ],
  "unknown_token": "UNKNOWN",
  "unknown_id_pattern": "^UNK-[0-9]{4,}$"
}

UNKNOWN_REGISTRY_FORMAT.v1.0.0.json

```
{
  "artifact_type": "UNKNOWN_REGISTRY_FORMAT",
  "format_version": "1.0.0",
  "required_fields": ["run_id", "unknowns"],
  "unknown_shape": {
    "unknown_id": "string",
    "title": "string",
    "question": "string",
    "blocking": "boolean",
    "created_by": "string",
    "created_at": "ISO date-time",
    "references": "array<string> (spec refs or doc refs)"
  }
}
```

Standards Mechanics

Standards Mechanics

- **Standards index contract**
- **Applicability map contract**
- **Constraint packs contract**
- **Resolved standards snapshot format**
- **Resolver order rules** (deterministic)

STANDARDS_INDEX.v1.0.0.json

```
{
  "artifact_type": "STANDARDS_INDEX",
  "standards_index_version": "1.0.0",
  "std_id_pattern": "^STD-[A-Z]{2,6}-[0-9]{2,3}$",
  "required_fields_per_standard": [
    "std_id",
    "title",
    "scope",
    "entry_path",
    "status"
  ],
  "standards": [
    {
      "std_id": "STD-NAM-01",
      "title": "Naming & IDs",
      "scope": "Global",
      "entry_path": "axion/system_docs/STD/STD-NAM-01.md",
      "status": "locked"
    },
    {
      "std_id": "STD-STR-01",
      "title": "Repo & Storage Layout",
      "scope": "Global",
      "entry_path": "axion/system_docs/STD/STD-STR-01.md",
      "status": "locked"
    },
    {
      "std_id": "STD-SPEC-01",
      "title": "Canonical Spec Integrity",
      "scope": "Canonical",
      "entry_path": "axion/system_docs/STD/STD-SPEC-01.md",
      "status": "locked"
    },
    {
      "std_id": "STD-SEC-01",
      "title": "Baseline Security",
      "scope": "Security",
      "entry_path": "axion/system_docs/STD/STD-SEC-01.md",
      "status": "locked"
    },
    {
      "std_id": "STD-VER-01",
      "title": "Verification & Proof",
      "scope": "Verification",
```

"entry_path": "axion/system_docs/STD/STD-VER-01.md",
"status": "locked"
}
],
"notes": "Index is authoritative inventory. Applicability is determined by APPLICABILITY_MAP."
}

APPLICABILITY_MAP.v1.0.0.json

```
{
  "artifact_type": "APPLICABILITY_MAP",
  "applicability_map_version": "1.0.0",

  "defaults": ["STD-NAM-01", "STD-STR-01", "STD-SPEC-01", "STD-VER-01"],

  "by_routing": {
    "category": {
      "application": [],
      "service": [],
      "infrastructure": [],
      "library_sdk": []
    },
    "audience_context": {
      "consumer": [],
      "internal": [],
      "developer": []
    },
    "build_target": {
      "new": [],
      "existing": []
    },
    "skill_level": {
      "beginner": [],
      "intermediate": [],
      "expert": []
    }
  },

  "by_gate": {
    "auth.required=true": ["STD-SEC-01"],
    "data.enabled=true": ["STD-SEC-01"],
    "integrations.enabled=true": ["STD-SEC-01"]
  },

  "resolution_rules": [
    "Start with defaults",
    "Add by_routing matches",
    "Add by_gate matches",
    "Deduplicate while preserving resolver order",
    "Fail if any referenced std_id is not present in STANDARDS_INDEX"
  ]
}
```

CONSTRAINT_PACKS.v1.0.0.json

```
{
  "artifact_type": "CONSTRAINT_PACKS",
  "constraint_packs_version": "1.0.0",
  "pack_id_pattern": "^CP-[A-Z0-9_]+-[0-9]{2,3}$",
  "packs": [
    {
      "pack_id": "CP-NAMING-01",
      "title": "Naming + Versioning Constraints",
      "std_ids": ["STD-NAM-01"],
      "constraints": [
        { "constraint_id": "NAM-001", "rule": "All compiled artifact filenames must be versioned:
*.vX.Y.Z.json" },
        { "constraint_id": "NAM-002", "rule": "All spec refs must match
ID_FORMAT_RULES.spec_ids.ref_pattern" }
      ]
    },
    {
      "pack_id": "CP-STORAGE-01",
      "title": "Storage & Immutability Constraints",
      "std_ids": ["STD-STR-01"],
      "constraints": [
        { "constraint_id": "STR-001", "rule": "Per-run outputs must live under axion/runs/{RUN_ID}/
only" },
        { "constraint_id": "STR-002", "rule": "Compiled contracts must live under axion/artifacts/
only" }
      ]
    },
    {
      "pack_id": "CP-CANONICAL-01",
      "title": "Canonical Spec Constraints",
      "std_ids": ["STD-SPEC-01"],
      "constraints": [
        { "constraint_id": "SPEC-001", "rule": "All refs must resolve within the canonical spec
payload" },
        { "constraint_id": "SPEC-002", "rule": "Spec IDs must be globally unique across buckets" }
      ]
    },
    {
      "pack_id": "CP-VERIFY-01",
      "title": "Verification + Proof Constraints",
      "std_ids": ["STD-VER-01"],
      "constraints": [
        { "constraint_id": "VER-001", "rule": "Every gate emits a Gate Report
(GATE_REPORT_FORMAT)" },
```

{ "constraint_id": "VER-002", "rule": "Proof ledger is append-only and references proof artifacts" }
]
}
]
}

RESOLVED_STANDARDS_SNAPSHOT_FORMAT.v1.0.0.json

```
{
  "artifact_type": "RESOLVED_STANDARDS_SNAPSHOT_FORMAT",
  "format_version": "1.0.0",
  "required_fields": [
    "run_id",
    "routing_snapshot",
    "gate_snapshot",
    "resolved_std_ids",
    "resolved_constraint_pack_ids",
    "resolver_version",
    "resolver_order_version"
  ],
  "shape": {
    "run_id": "string",
    "routing_snapshot": "object",
    "gate_snapshot": "object",
    "resolved_std_ids": "array<string>",
    "resolved_constraint_pack_ids": "array<string>",
    "resolver_version": "string",
    "resolver_order_version": "string",
    "notes": "string"
  }
}
```

STANDARDS_RESOLVER_ORDER_RULES.v1.0.0.json

```
{
  "artifact_type": "STANDARDS_RESOLVER_ORDER_RULES",
  "order_version": "1.0.0",
  "rules": [
    "Resolver order is deterministic and stable within a version.",
    "Defaults apply first, then routing-based additions, then gate-based additions.",
    "Deduplication preserves first occurrence order.",
    "If two rules add the same std_id, the earliest applied rule wins ordering."
  ],
  "priority_order": [
    "defaults",
    "by_routing.category",
    "by_routing.audience_context",
    "by_routing.build_target",
    "by_routing.skill_level",
    "by_gate"
  ]
}
```

Template System Mechanics

Template System Mechanics (format + enforcement only)

- **Template index/registry contract**
- **Template selection result format**
- **Render output/envelope format**
- **Template completeness rules**
- **Placeholder/derive syntax rules** (the allowed forms)

TEMPLATE_INDEX_REGISTRY.v1.0.0.json

TEMPLATE_SELECTION_RESULT_FORMAT.v1.0.0.json

```
{
  "artifact_type": "TEMPLATE_SELECTION_RESULT_FORMAT",
  "format_version": "1.0.0",
  "required_fields": [
    "run_id",
    "routing_snapshot",
    "gate_snapshot",
    "selection_version",
    "selected",
    "omitted",
    "ordering"
  ],
  "selected_item_shape": {
    "template_id": "string",
    "template_version": "string",
    "file_path": "string",
    "output_path": "string",
    "reason": "string"
  },
  "omitted_item_shape": {
    "template_id": "string",
    "reason": "string"
  },
  "ordering_rules": [
    "Ordering must be deterministic and recorded in ordering[].",
    "Default ordering: registry order as stored, unless registry specifies an ordering_rule field."
  ]
}
```

TEMPLATE_RENDER_OUTPUT_FORMAT.v1.0.0.json

```
{
  "artifact_type": "TEMPLATE_RENDER_OUTPUT_FORMAT",
  "render_format_version": "1.0.0",

  "rendered_doc_envelope_required_fields": [
    "run_id",
    "doc_id",
    "template_id",
    "template_version",
    "rendered_version",
    "status",
    "created_at",
    "source_refs",
    "content_md"
  ],

  "rendered_doc_envelope_shape": {
    "run_id": "string",
    "doc_id": "string",
    "template_id": "string",
    "template_version": "string",
    "rendered_version": "string",
    "status": "draft|locked",
    "created_at": "ISO date-time",
    "source_refs": "array<{name:string, version:string, path?:string, hash?:string}>",
    "content_md": "string"
  },

  "storage_layout": {
    "run_docs_root": "axion/runs/{{RUN_ID}}/docs_rendered/",
    "doc_paths": [
      "axion/runs/{{RUN_ID}}/docs_rendered/{{DOC_ID}}/{{DOC_ID}}.md",
      "axion/runs/{{RUN_ID}}/docs_rendered/{{DOC_ID}}/envelope.json"
    ]
  },

  "versioning_rules": [
    "rendered_version is scoped to the run (not global).",
    "If a rendered doc is re-rendered within the same run, it must write a new rendered_version
file or be recorded as an append-only update record."
  ]
}
```

TEMPLATE_COMPLETENESS_RULES
.v1.0.0.json

```
{
  "artifact_type": "TEMPLATE_COMPLETENESS_RULES",
  "rules_version": "1.0.0",

  "global_checks": [
    {
      "rule_id": "TMP-05-001",
      "description": "No unresolved placeholders remain ({{...}}) in rendered content",
      "check": "no_placeholder_tokens",
      "error_code": "UNRESOLVED_PLACEHOLDER",
      "severity": "error"
    },
    {
      "rule_id": "TMP-05-002",
      "description": "Rendered doc must contain required top-level sections in the locked order",
      "check": "required_sections_present_and_ordered",
      "error_code": "MISSING_OR_MISORDERED_SECTION",
      "severity": "error"
    },
    {
      "rule_id": "TMP-05-003",
      "description": "Cross-references must resolve (refs must exist in Canonical Spec or declared
registries)",
      "check": "cross_refs_resolve",
      "error_code": "UNRESOLVED_REF",
      "severity": "error"
    }
  ],

  "required_top_level_sections_in_order": [
    "Header Block",
    "Purpose",
    "Inputs Required",
    "Required Fields",
    "Optional Fields",
    "Rules",
    "Output Format",
    "Cross-References",
    "Skill Level Requiredness Rules",
    "Unknown Handling",
    "Completeness Gate"
  ],

  "per_template_overrides": [],
```

"notes": "Per-template required fields/headings live in templates or registry; this artifact enforces universal checks."
}

PLACEHOLDER_DERIVE_SYNTAX_RULES.v1.0.0.json

```
{
  "artifact_type": "PLACEHOLDER_DERIVE_SYNTAX_RULES",
  "rules_version": "1.0.0",

  "placeholder_tokens": {
    "open": "{{",
    "close": "}}"
  },

  "allowed_sources": [
    "routing",
    "project",
    "intent",
    "spec",
    "data",
    "auth",
    "integrations",
    "nfr",
    "canonical",
    "standards",
    "plan",
    "acceptance",
    "run"
  ],

  "allowed_forms": [
    {
      "form": "{{path.to.value}}",
      "description": "Direct path lookup from allowed source objects",
      "examples": ["{{routing.category}}", "{{project.project_name}}"]
    },
    {
      "form": "{{canonical.spec_index.screen[0].title}}",
      "description": "Canonical spec lookup with explicit indices",
      "examples": ["{{canonical.spec_index.entity[0].fields[0].name}}"]
    },
    {
      "form": "{{standards.std[STD-SEC-01].summary}}",
      "description": "Standards snapshot lookup by std_id",
      "examples": ["{{standards.std[STD-SEC-01].rules[0]}}"]
    },
    {
      "form": "{{derive:FUNCTION(ARG1,ARG2)}}",
      "description": "Deterministic derive functions (no network, no randomness)",
```

"examples": ["{{derive:FEATURE_TO_WORKFLOWS(FEAT_001)}}"]
}
],

"flags": [
{
"flag": "OPTIONAL",
"meaning": "If missing, render as empty and do not fail completeness gate"
},
{
"flag": "UNKNOWN_ALLOWED",
"meaning": "If missing, may render UNKNOWN token and must record unknown_id when unknowns policy is enabled"
}
],

"prohibitions": [
"No freeform code execution inside placeholders",
"No network calls to resolve placeholders",
"No implicit derivations not declared via derive:FUNCTION"
],

"notes": "Renderer must validate placeholder forms against these rules before rendering."
}

Planning Mechanics

Planning Mechanics

- **Work breakdown format**
- **Acceptance map format**
- **Build plan format**
- **Coverage rules** (what must be covered for a run to be “complete”)

WORK_BREAKDOWN_FORMAT.v1.0.0.json

```
{
  "artifact_type": "WORK_BREAKDOWN_FORMAT",
  "format_version": "1.0.0",
  "required_fields": [
    "run_id",
    "spec_schema_version",
    "canonical_spec_ref",
    "units"
  ],
  "unit_shape": {
    "unit_id": "string",
    "title": "string",
    "domain": "string|optional",
    "scope_refs": "array<string> (spec refs)",
    "depends_on": "array<string> (unit_ids)",
    "outputs": "array<string> (planned artifacts/docs/code outputs)",
    "risk_flags": "array<string>",
    "status": "planned|in_progress|done"
  },
  "rules": [
    "unit_id must be unique within the work breakdown",
    "scope_refs must be valid spec refs (ID_FORMAT_RULES.spec_ids)",
    "depends_on must reference existing unit_id values"
  ]
}
```

ACCEPTANCE_MAP_FORMAT.v1.0.0.json

```
{
  "artifact_type": "ACCEPTANCE_MAP_FORMAT",
  "format_version": "1.0.0",
  "required_fields": [
    "run_id",
    "work_breakdown_ref",
    "acceptance"
  ],
  "acceptance_item_shape": {
    "acceptance_id": "string",
    "unit_id": "string",
    "statement": "string",
    "evidence_required": "array<string> (proof types)",
    "covers_refs": "array<string> (spec refs)",
    "test_refs": "array<string> (optional spec refs to test_case items)"
  },
  "rules": [
    "acceptance_id must be unique",
    "unit_id must exist in referenced WORK_BREAKDOWN.units",
    "covers_refs must be valid spec refs"
  ]
}
```

BUILD_PLAN_FORMAT.v1.0.0.json

```
{
  "artifact_type": "BUILD_PLAN_FORMAT",
  "format_version": "1.0.0",
  "required_fields": [
    "run_id",
    "routing_snapshot",
    "work_sequence",
    "unit_execution_rules"
  ],
  "work_sequence_item_shape": {
    "unit_id": "string",
    "order": "number",
    "why": "string",
    "gates_before": "array<string> (gate_ids)",
    "gates_after": "array<string> (gate_ids)"
  },
  "unit_execution_rules_shape": {
    "stop_on_fail": "boolean",
    "retry_policy": "object|optional",
    "allowed_tools": "array<string>|optional"
  },
  "rules": [
    "work_sequence.order must be strictly increasing",
    "each work_sequence.unit_id must exist in WORK_BREAKDOWN.units",
    "gate_ids referenced must exist in pipeline gate definitions"
  ]
}
```

COVERAGE_RULES.v1.0.0.json

```
{
  "artifact_type": "COVERAGE_RULES",
  "rules_version": "1.0.0",

  "coverage_targets": [
    "screen",
    "endpoint",
    "entity",
    "business_rule",
    "component",
    "acceptance_criterion",
    "test_case",
    "flow"
  ],

  "required_by_category": {
    "application": {
      "must_have": ["screen", "flow", "component", "acceptance_criterion"],
      "should_have": ["entity", "endpoint", "test_case", "business_rule"]
    },
    "service": {
      "must_have": ["endpoint", "flow", "acceptance_criterion"],
      "should_have": ["entity", "test_case", "business_rule", "component", "screen"]
    },
    "infrastructure": {
      "must_have": ["flow", "acceptance_criterion"],
      "should_have": ["endpoint", "entity", "test_case", "business_rule", "component", "screen"]
    },
    "library_sdk": {
      "must_have": ["endpoint", "flow", "acceptance_criterion"],
      "should_have": ["test_case", "business_rule", "entity", "component", "screen"]
    }
  },

  "completeness_rule": {
    "definition": "A run is complete when all must_have buckets for routing.category have at least
one item OR an explicit N/A waiver is recorded.",
    "na_waiver_required": true
  },

  "na_waiver_format": {
    "required_fields": ["bucket", "reason", "approved_by"],
    "bucket": "string",
    "reason": "string",
```

"approved_by": "string"
}
}

Pipeline Orchestration Mechanics

Pipeline Orchestration Mechanics

- **Pipeline definition** (stages + order)
- **Stage IO contracts** (consumes/produces)
- **Gate model** (gate_id, pass condition, evidence refs)
- **Stage report format**
- **Run manifest format**

PIPELINE_DEFINITION.v1.0.0.json

{
  "artifact_type": "PIPELINE_DEFINITION",
  "pipeline_version": "1.0.0",
  "pipeline_id": "AXION_PIPELINE_V1",

  "stage_order": [
    "S1_INGEST_NORMALIZE",
    "S2_VALIDATE_INTAKE",
    "S3_BUILD_CANONICAL",
    "S4_VALIDATE_CANONICAL",
    "S5_RESOLVE_STANDARDS",
    "S6_SELECT_TEMPLATES",
    "S7_RENDER_DOCS",
    "S8_BUILD_PLAN",
    "S9_VERIFY_PROOF",
    "S10_PACKAGE"
  ],

  "stages": [
    {
      "stage_id": "S1_INGEST_NORMALIZE",
      "title": "Ingest + Normalize",
      "consumes": ["INTAKE_SUBMISSION"],
      "produces": ["INTAKE_SUBMISSION_RECORD", "NORMALIZED_INPUT_RECORD"],
      "gates": []
    },
    {
      "stage_id": "S2_VALIDATE_INTAKE",
      "title": "Validate Intake",
      "consumes": ["NORMALIZED_INPUT_RECORD", "INTAKE_SCHEMA",
"INTAKE_RULESET", "ENUM_REGISTRY"],
      "produces": ["INTAKE_VALIDATION_RESULT"],
      "gates": ["G1_INTAKE_VALIDITY"]
    },
    {
      "stage_id": "S3_BUILD_CANONICAL",
      "title": "Build Canonical Spec",
      "consumes": ["NORMALIZED_INPUT_RECORD", "INTAKE_VALIDATION_RESULT",
"CANONICAL_SPEC_SCHEMA"],
      "produces": ["CANONICAL_SPEC"],
      "gates": []
    },
    {
      "stage_id": "S4_VALIDATE_CANONICAL",

"title": "Validate Canonical Spec",
"consumes": ["CANONICAL_SPEC", "CANONICAL_VALIDATION_RULES"],
"produces": ["GATE_REPORT"],
"gates": ["G2_CANONICAL_INTEGRITY"]
},
{
"stage_id": "S5_RESOLVE_STANDARDS",
"title": "Resolve Standards",
"consumes": ["CANONICAL_SPEC", "STANDARDS_INDEX", "APPLICABILITY_MAP", "CONSTRAINT_PACKS"],
"produces": ["RESOLVED_STANDARDS_SNAPSHOT"],
"gates": ["G3_STANDARDS_RESOLVED"]
},
{
"stage_id": "S6_SELECT_TEMPLATES",
"title": "Select Templates",
"consumes": ["CANONICAL_SPEC", "RESOLVED_STANDARDS_SNAPSHOT", "TEMPLATE_INDEX_REGISTRY"],
"produces": ["TEMPLATE_SELECTION_RESULT"],
"gates": ["G4_TEMPLATE_SELECTION"]
},
{
"stage_id": "S7_RENDER_DOCS",
"title": "Render Docs",
"consumes": ["TEMPLATE_SELECTION_RESULT", "CANONICAL_SPEC", "RESOLVED_STANDARDS_SNAPSHOT", "PLACEHOLDER_DERIVE_SYNTAX_RULES"],
"produces": ["RENDERED_DOCS_SET"],
"gates": ["G5_TEMPLATE_COMPLETENESS"]
},
{
"stage_id": "S8_BUILD_PLAN",
"title": "Build Plan",
"consumes": ["CANONICAL_SPEC", "RENDERED_DOCS_SET", "COVERAGE_RULES"],
"produces": ["WORK_BREAKDOWN", "ACCEPTANCE_MAP", "BUILD_PLAN"],
"gates": ["G6_PLAN_COVERAGE"]
},
{
"stage_id": "S9_VERIFY_PROOF",
"title": "Verify + Proof",
"consumes": ["BUILD_PLAN", "ACCEPTANCE_MAP", "VERIFICATION_COMMAND_POLICY", "PROOF_TYPES_RULES"],
"produces": ["GATE_REPORTS", "PROOF_LEDGER"],
"gates": ["G7_VERIFICATION"]
},

```
    {
      "stage_id": "S10_PACKAGE",
      "title": "Package",
      "consumes": ["FOLDER_TREE_SPEC", "KIT_MANIFEST_FORMAT",
"PACKAGING_MANIFEST_FORMAT", "RUN_MANIFEST"],
      "produces": ["PACKAGING_MANIFEST", "BUNDLE_OUTPUT"],
      "gates": ["G8_PACKAGE_INTEGRITY"]
    }
  ],

  "gate_ids": [
    "G1_INTAKE_VALIDITY",
    "G2_CANONICAL_INTEGRITY",
    "G3_STANDARDS_RESOLVED",
    "G4_TEMPLATE_SELECTION",
    "G5_TEMPLATE_COMPLETENESS",
    "G6_PLAN_COVERAGE",
    "G7_VERIFICATION",
    "G8_PACKAGE_INTEGRITY"
  ]
}
```

STAGE_IO_CONTRACTS.v1.0.0.json

{
  "artifact_type": "STAGE_IO_CONTRACTS",
  "contracts_version": "1.0.0",
  "contracts": [
    {
      "stage_id": "S1_INGEST_NORMALIZE",
      "consumes_required": ["INTAKE_SUBMISSION"],
      "produces_required": ["INTAKE_SUBMISSION_RECORD",
"NORMALIZED_INPUT_RECORD"]
    },
    {
      "stage_id": "S2_VALIDATE_INTAKE",
      "consumes_required": ["NORMALIZED_INPUT_RECORD", "INTAKE_SCHEMA",
"INTAKE_RULESET", "ENUM_REGISTRY"],
      "produces_required": ["INTAKE_VALIDATION_RESULT"]
    },
    {
      "stage_id": "S3_BUILD_CANONICAL",
      "consumes_required": ["NORMALIZED_INPUT_RECORD",
"INTAKE_VALIDATION_RESULT", "CANONICAL_SPEC_SCHEMA"],
      "produces_required": ["CANONICAL_SPEC"]
    },
    {
      "stage_id": "S4_VALIDATE_CANONICAL",
      "consumes_required": ["CANONICAL_SPEC", "CANONICAL_VALIDATION_RULES"],
      "produces_required": ["GATE_REPORT"]
    },
    {
      "stage_id": "S5_RESOLVE_STANDARDS",
      "consumes_required": ["CANONICAL_SPEC", "STANDARDS_INDEX",
"APPLICABILITY_MAP", "CONSTRAINT_PACKS"],
      "produces_required": ["RESOLVED_STANDARDS_SNAPSHOT"]
    },
    {
      "stage_id": "S6_SELECT_TEMPLATES",
      "consumes_required": ["CANONICAL_SPEC", "RESOLVED_STANDARDS_SNAPSHOT",
"TEMPLATE_INDEX_REGISTRY"],
      "produces_required": ["TEMPLATE_SELECTION_RESULT"]
    },
    {
      "stage_id": "S7_RENDER_DOCS",
      "consumes_required": ["TEMPLATE_SELECTION_RESULT", "CANONICAL_SPEC",
"RESOLVED_STANDARDS_SNAPSHOT", "PLACEHOLDER_DERIVE_SYNTAX_RULES"],
      "produces_required": ["RENDERED_DOCS_SET"]

```
    },
    {
      "stage_id": "S8_BUILD_PLAN",
      "consumes_required": ["CANONICAL_SPEC", "RENDERED_DOCS_SET",
"COVERAGE_RULES"],
      "produces_required": ["WORK_BREAKDOWN", "ACCEPTANCE_MAP", "BUILD_PLAN"]
    },
    {
      "stage_id": "S9_VERIFY_PROOF",
      "consumes_required": ["BUILD_PLAN", "ACCEPTANCE_MAP",
"VERIFICATION_COMMAND_POLICY", "PROOF_TYPES_RULES"],
      "produces_required": ["GATE_REPORTS", "PROOF_LEDGER"]
    },
    {
      "stage_id": "S10_PACKAGE",
      "consumes_required": ["FOLDER_TREE_SPEC", "KIT_MANIFEST_FORMAT",
"PACKAGING_MANIFEST_FORMAT", "RUN_MANIFEST"],
      "produces_required": ["PACKAGING_MANIFEST", "BUNDLE_OUTPUT"]
    }
  ]
}
```

GATE_MODEL.v1.0.0.json

```
{
  "artifact_type": "GATE_MODEL",
  "model_version": "1.0.0",
  "gate_shape": {
    "gate_id": "string",
    "stage_id": "string",
    "inputs": "array<string>",
    "pass_condition": "string",
    "evidence_required": "array<string>",
    "failure_codes": "array<string>"
  },
  "notes": "Each executed gate emits a Gate Report and proof ledger entries."
}
```

GATE_DEFINITIONS.v1.0.0.json

{
  "artifact_type": "GATE_DEFINITIONS",
  "definitions_version": "1.0.0",
  "gates": [
    {
      "gate_id": "G1_INTAKE_VALIDITY",
      "stage_id": "S2_VALIDATE_INTAKE",
      "inputs": ["INTAKE_SCHEMA", "INTAKE_RULESET", "NORMALIZED_INPUT_RECORD"],
      "pass_condition": "INTAKE_VALIDATION_RESULT.result.status == 'pass'",
      "evidence_required": ["validation_report"],
      "failure_codes": ["REQUIRED", "DEPENDENCY_MISSING",
"INVALID_ENUM_FOR_CONTEXT"]
    },
    {
      "gate_id": "G2_CANONICAL_INTEGRITY",
      "stage_id": "S4_VALIDATE_CANONICAL",
      "inputs": ["CANONICAL_SPEC", "CANONICAL_VALIDATION_RULES",
"REFERENCE_RESOLUTION_RULES"],
      "pass_condition": "Gate report status == 'pass'",
      "evidence_required": ["gate_report"],
      "failure_codes": ["INVALID_REF_FORMAT", "UNRESOLVED_REF", "DUPLICATE_VALUE",
"DISALLOWED_EDGE"]
    },
    {
      "gate_id": "G3_STANDARDS_RESOLVED",
      "stage_id": "S5_RESOLVE_STANDARDS",
      "inputs": ["STANDARDS_INDEX", "APPLICABILITY_MAP", "CONSTRAINT_PACKS"],
      "pass_condition": "RESOLVED_STANDARDS_SNAPSHOT.resolved_std_ids.length >= 1",
      "evidence_required": ["resolved_standards_snapshot"],
      "failure_codes": ["UNKNOWN_STANDARD", "RESOLUTION_FAILED"]
    },
    {
      "gate_id": "G6_PLAN_COVERAGE",
      "stage_id": "S8_BUILD_PLAN",
      "inputs": ["CANONICAL_SPEC", "COVERAGE_RULES"],
      "pass_condition": "All must_have buckets satisfied OR N/A waivers recorded",
      "evidence_required": ["coverage_report"],
      "failure_codes": ["COVERAGE_MISSING"]
    },
    {
      "gate_id": "G8_PACKAGE_INTEGRITY",
      "stage_id": "S10_PACKAGE",
      "inputs": ["PACKAGING_MANIFEST", "BUNDLE_OUTPUT"],
      "pass_condition": "Manifest hashes match bundle bytes",

"evidence_required": ["packaging_manifest"],
"failure_codes": ["HASH_MISMATCH", "MISSING_FILE"]
}
]
}

STAGE_REPORT_FORMAT.v1.0.0.json

```
{
  "artifact_type": "STAGE_REPORT_FORMAT",
  "stage_report_version": "1.0.0",
  "required_fields": [
    "run_id",
    "stage_id",
    "status",
    "started_at",
    "ended_at",
    "consumed",
    "produced",
    "gate_reports"
  ],
  "shape": {
    "run_id": "string",
    "stage_id": "string",
    "status": "pass|fail|skipped",
    "started_at": "ISO date-time",
    "ended_at": "ISO date-time",
    "consumed": "array<{artifact:string, version?:string, path?:string, hash?:string}>",
    "produced": "array<{artifact:string, version?:string, path?:string, hash?:string}>",
    "gate_reports": "array<{gate_id:string, status:pass|fail, report_path?:string}>",
    "notes": "string"
  }
}
```

RUN_MANIFEST_FORMAT.v1.0.0.json

```
{
  "artifact_type": "RUN_MANIFEST_FORMAT",
  "format_version": "1.0.0",
  "required_fields": [
    "run_id",
    "pipeline_id",
    "pipeline_version",
    "config_version",
    "config_hash",
    "routing_snapshot",
    "active_versions",
    "policy_snapshot",
    "started_at",
    "ended_at",
    "status",
    "stage_reports",
    "outputs"
  ],
  "shape": {
    "run_id": "string",
    "pipeline_id": "string",
    "pipeline_version": "string",
    "config_version": "string",
    "config_hash": "sha256",
    "routing_snapshot": "object",
    "active_versions": "object",
    "policy_snapshot": "object",
    "started_at": "ISO date-time",
    "ended_at": "ISO date-time",
    "status": "pass|fail",
    "stage_reports": "array<string> (paths)",
    "outputs": "object (artifact->path map)"
  }
}
```

Verification & Proof Mechanics

Verification & Proof Mechanics

- **Gate report format**
- **Proof ledger format**
- **Proof types rules**
- **Verification command policy**
- **Completion criteria** (unit done + run done)

GATE_REPORT_FORMAT.v1.0.0.json

```
{
  "artifact_type": "GATE_REPORT_FORMAT",
  "format_version": "1.0.0",
  "required_fields": [
    "run_id",
    "gate_id",
    "stage_id",
    "status",
    "started_at",
    "ended_at",
    "failures",
    "warnings",
    "proof_refs"
  ],
  "shape": {
    "run_id": "string",
    "gate_id": "string",
    "stage_id": "string",
    "status": "pass|fail",
    "started_at": "ISO date-time",
    "ended_at": "ISO date-time",
    "failures": "array<RuleHit>",
    "warnings": "array<RuleHit>",
    "proof_refs": "array<ProofRef>",
    "notes": "string"
  },
  "RuleHit": {
    "rule_id": "string",
    "error_code": "string",
    "severity": "error|warning",
    "message": "string",
    "pointer_paths": "array<string>",
    "details": "object"
  },
  "ProofRef": {
    "proof_id": "string",
    "type": "string",
    "path": "string",
    "sha256": "string"
  }
}
```

PROOF_LEDGER_FORMAT.v1.0.0.json

```
{
  "artifact_type": "PROOF_LEDGER_FORMAT",
  "format_version": "1.0.0",
  "storage": {
    "format": "jsonl",
    "path": "axion/runs/{{RUN_ID}}/proofs/proof_ledger.jsonl"
  },
  "required_fields_per_entry": [
    "proof_id",
    "run_id",
    "stage_id",
    "gate_id",
    "timestamp",
    "type",
    "summary",
    "artifact_paths",
    "sha256"
  ],
  "entry_shape": {
    "proof_id": "string",
    "run_id": "string",
    "stage_id": "string",
    "gate_id": "string|optional",
    "timestamp": "ISO date-time",
    "type": "string",
    "summary": "string",
    "artifact_paths": "array<string>",
    "sha256": "string",
    "metadata": "object"
  },
  "append_only_rules": [
    "Ledger is append-only. No edits or deletions within a run.",
    "If a correction is needed, append a new entry referencing the prior proof_id."
  ]
}
```

PROOF_TYPES_RULES.v1.0.0.json

```
{
  "artifact_type": "PROOF_TYPES_RULES",
  "rules_version": "1.0.0",
  "proof_types": [
    {
      "type": "validation_report",
      "description": "Schema/rules validation output",
      "required_fields": ["path", "sha256"]
    },
    {
      "type": "gate_report",
      "description": "Gate pass/fail report",
      "required_fields": ["path", "sha256"]
    },
    {
      "type": "hash_manifest",
      "description": "Hash inventory / packaging manifest",
      "required_fields": ["path", "sha256"]
    },
    {
      "type": "command_output",
      "description": "Captured output of a verification command",
      "required_fields": ["path", "sha256", "command"]
    },
    {
      "type": "log_excerpt",
      "description": "Captured log snippet for evidence",
      "required_fields": ["path", "sha256"]
    }
  ],
  "rules": [
    "Every proof_ref.type must be listed in proof_types[].type",
    "Every proof_ref.sha256 must match the referenced file bytes"
  ]
}
```

VERIFICATION_COMMAND_POLICY.v1.0.0.json

```
{
  "artifact_type": "VERIFICATION_COMMAND_POLICY",
  "policy_version": "1.0.0",
  "rules": [
    "Only whitelisted commands may be executed as verification steps.",
    "All command outputs must be captured to runs/{{RUN_ID}}/proofs/ and hashed.",
    "No network calls are allowed during verification.",
    "Commands must be deterministic (no prompts, no interactive input)."
  ],
  "whitelist": [
    {
      "id": "CMD-001",
      "name": "unit_tests",
      "command": "npm test",
      "capture_path": "axion/runs/{{RUN_ID}}/proofs/unit_tests.out.txt"
    },
    {
      "id": "CMD-002",
      "name": "lint",
      "command": "npm run lint",
      "capture_path": "axion/runs/{{RUN_ID}}/proofs/lint.out.txt"
    }
  ]
}
```

COMPLETION_CRITERIA.v1.0.0.json

```
{
  "artifact_type": "COMPLETION_CRITERIA",
  "criteria_version": "1.0.0",

  "unit_done": {
    "required": [
      "unit_status == 'done'",
      "all_acceptance_for_unit_passed == true",
      "required_proofs_present == true"
    ],
    "notes": "Unit completion is tied to acceptance criteria and proof presence."
  },

  "run_done": {
    "required": [
      "all_required_gates_passed == true",
      "coverage_must_have_satisfied == true",
      "packaging_manifest_verified == true",
      "bundle_output_created == true"
    ],
    "notes": "Run completion requires gates + coverage + packaging integrity."
  },

  "required_gates": [
    "G1_INTAKE_VALIDITY",
    "G2_CANONICAL_INTEGRITY",
    "G3_STANDARDS_RESOLVED",
    "G6_PLAN_COVERAGE",
    "G8_PACKAGE_INTEGRITY"
  ]
}
```